

Esame OS Internals (Cabodi)

Data: 11/07/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Memoria Virtuale e Paginazione)

Contesto: Un sistema informatico ha le seguenti caratteristiche:

- Dimensione della memoria fisica: 512 MB
- Spazio degli indirizzi virtuali: indirizzi a 32 bit
- Dimensioni pagina: 8 KB
- Utilizza una tabella di pagine a due livelli
- Implementa un algoritmo di sostituzione delle pagine least recently used (LRU)

Q1.A) Rispondere alle domande seguenti (Vero/Falso o Scelta Multipla):

1. La presenza di una TLB garantisce sempre un tempo di accesso alla memoria più rapido rispetto a un sistema senza TLB?
2. È possibile eseguire il mapping di due indirizzi virtuali diversi allo stesso indirizzo fisico in un sistema che utilizza il paging?
3. A un processo sono allocati più frame fisici rispetto alle dimensioni del working set (per un determinato tempo delta): questo ridurrà sempre il tasso di errore della pagina a zero?
4. È possibile progettare una tabella di pagine dimensionata dinamicamente in base al numero di pagine attualmente utilizzate da un processo, riducendo così al minimo il carico di memoria?
5. L'utilizzo di una pagina di dimensioni maggiori può comportare un utilizzo più efficiente della TLB nonostante il potenziale aumento della frammentazione interna?
6. Se viene utilizzato l'algoritmo di sostituzione pagine LRU, garantisce il minor numero possibile di page fault?

(Nota: Per ognuna delle risposte alle domande precedenti, è richiesta una motivazione/spiegazione).

Q1.B) Esercizio LRU:

Data la seguente stringa di riferimento a memoria per un processo (indirizzamento dei byte, con indirizzi espressi in codice esadecimale) e le rispettive operazioni di lettura(R)/scrittura (W):

R 1F40, W 3E20, R 5D18, W 7C9E, R 2AF3, W 6B2C, R 13B2, W 24AB, R 54BC, W 12AF

Si suppongano 3 frame disponibili e si utilizzi l'algoritmo di sostituzione della pagina LRU.

1. Calcolare la stringa di riferimenti di pagina e indicare il numero di pagina corrispondente per ogni riferimento a memoria.

2. Indicare esplicitamente per ogni frame allocato e tempo/riferimento, il numero di pagina e se si verifica un page fault.
 3. Calcolare il numero totale di page fault che si verificano durante questa sequenza di riferimenti alla memoria.
-

Domanda 2 (File System e Inodes)

Contesto: Consider a Unix-like file system based on inodes, with 15 pointers (12 direct, 1 single indirect, 1 double indirect, and 1 triple indirect). Pointers have a size of 32 bits, and disk blocks have a size of 4 KB. The file system resides on a disk partition where 1 TB is reserved for data blocks. The extra space reserved for metadata (including index blocks) can be neglected for the purposes of this exercise.

- Q2.A)** Calculate the maximum file size supported by this file system.
- Q2.A)** Given a binary file of size 25,600 KB, compute exactly how many index blocks and data blocks the file is using. Also compute the internal fragmentation (for data blocks).
- Q2.A)** Suppose an operation `lseek(fd, offset, SEEK_END)` is called to position the file offset for a subsequent read/write operation. Given a binary file of size 10 GB and an offset of -1,048,576 bytes (1 MB) from the end of the file, compute the logical block number (relative to the file blocks, numbered starting at 0) to which the position is moved.
-

Domanda 3 (I/O, HDD, SSD e RAID)

Contesto: Si consideri un sistema informatico che utilizza una combinazione di unità disco rigido (HDD) e unità a stato solido (SSD) per l'archiviazione di massa. Il sistema incorpora anche vari dispositivi di I/O gestiti da un sottosistema di I/O.

Q3.A) Rispondere alle seguenti domande (e fornire spiegazioni/motivazioni):

1. È sempre più efficiente utilizzare l'accesso diretto alla memoria (DMA) per tutti i tipi di operazioni di I/O rispetto all'I/O basato su interrupt?
2. Il polling è un metodo più efficiente per la gestione delle operazioni di I/O rispetto all'utilizzo di interrupt in un ambiente di rete ad alta velocità?
3. È possibile che un'operazione di I/O su un SSD abbia sempre una latenza inferiore rispetto alla stessa operazione su un HDD?
4. È possibile che un SSD soffra di problemi di frammentazione simili a quelli riscontrati dagli HDD?
5. Uno scheduler I/O ben progettato può migliorare significativamente le prestazioni di una configurazione RAID 5 rispetto a uno scheduler I/O mal progettato? (*RAID 5: block interleaved distributed parity, è caratterizzato da bit di parità ridondanti*)
6. Una configurazione RAID 0 (striping) può migliorare l'affidabilità dei dati rispetto a una configurazione HDD singolo?

2 SECTION 2: OS161

Domanda 4 (Indirizzamento OS161 e `load_elf`)

Contesto: Viene fornito un sistema OS161, in esecuzione su un simulatore MIPS `sys161` con 8 MB di memoria RAM. È noto che il processo P1 ha uno spazio di indirizzamento con `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2` con i seguenti valori: 0x200000, 0x100000, 0x4000, 0x8000, 3, 4.

Q4.A) Traduzione degli indirizzi:

Per ognuno dei seguenti indirizzi fisici, convertire in un indirizzo logico del kernel. Nel caso in cui l'indirizzo sia allocato a P1, convertirlo anche in un indirizzo user logico:

1. 0x100A00
2. 0x10A0F0
3. 0x206500
4. 0x202D00

Q4.B) Funzioni `load_elf`:

La funzione OS161 `load_elf` predispone lo spazio degli indirizzi (address space) di un nuovo processo user. In fasi diverse, `load_elf` chiama `load_segment`, `as_prepare_load`, `as_complete_load`, `as_define_region`. Rispondere alle seguenti domande associando la funzione corretta:

1. Quale funzione viene chiamata per allocare lo spazio degli indirizzi nella memoria fisica?
 2. Quale funzione legge il segmento di codice dal file elf?
 3. Quale funzione viene chiamata per impostare gli indirizzi logici e le dimensioni del codice e dei segmenti di dati?
 4. Quale funzione legge il segmento di dati dal file elf?
-

Domanda 5 (OS161 Lab4 - Processi e System Calls)

Contesto: Consider the framework of os161 lab4. Rispondere alle domande seguenti relative a lab4:

Q5.A) Domande a risposta multipla/Vero-Falso:

1. In lab4 un processo deve essere identificato da un pid. Il pid è associato a un processo in fase di bootstrap?
2. Considerare le funzioni `sys_exit` e `sys_waitpid` implementate in lab4. Sono chiamate di sistema che possono essere chiamate dai processi utente?
3. Si chiama `proc_destroy` all'interno della chiamata di sistema `waitpid`?
4. La funzione `common_prog` produce un crash quando mancano implementazione e utilizzo opportuno di `waitpid` (o funzioni correlate, realizzate in lab4)?
5. Considerare le funzioni `sys_exit` e `sys_waitpid` implementate in lab4. Sono già presenti nella versione base di OS161?
6. Si chiama `proc_destroy` all'interno della chiamata di sistema `exit`?

7. In lab4 un processo deve essere identificato da un pid. Un pid è un puntatore?
8. Si chiama `proc_destroy` direttamente dalla funzione `thread_exit`?